AN5990

LAN865x Linux® Driver Installation

1.0 INTRODUCTION

The LAN865x integrates a Media Access Controller (MAC) and an Ethernet PHY, allowing cost-effective microcontrollers, even those without an on-board MAC, to connect to 10BASE-T1S networks. The Ethernet MAC module supports 10 Mbps half-duplex Ethernet, compliant with the IEEE 802.3™ standard, and includes a 10BASE-T1S physical layer transceiver. Communication between the host and the MAC-PHY follows the OPEN Alliance 10BASE-T1x MAC-PHY Serial Interface (TC6) specification. The LAN865x's standard Serial Peripheral Interface (SPI) facilitates interfacing with almost any microcontroller, enabling Ethernet packet transfer and LAN865x control/status commands over a single serial interface. The SPI requires only four pins, offering a simpler hardware interface with fewer pins compared to MII or RMII.

1.1 Audience

The LAN865x is intended to be used for developing or testing 10BASE-T1S Ethernet network devices by persons with experience in this field of knowledge.

1.2 System Requirements

LAN8650/1 Revision	Supported Kernel Version
B0	6.12 onwards
B1	6.13 onwards

1.3 Supported Platforms

- ARM

1.4 Test Platform and Devices

Test platform:	Raspberry Pi® 4 Model B
Kernel version:	6.6.51 6.12.25
Tested device:	Two-Wire ETH Click boards™ [3] with LAN8651, Rev B1

1.5 References

The following sources should be referenced when using this application note:

[1] LAN8650
https://www.microchip.com/en-us/product/lan8650

[2] LAN8651
https://www.microchip.com/en-us/product/lan8651

[3] Two-Wire ETH Click board
https://www.mikroe.com/two-wire-eth-click

1.6 Abbreviations

Abbreviation	Definition
EVB	Evaluation Board
LKM	Loadable Kernel Module
MAC	Media Access Control
MII	Media Independent Interface
PLCA	Physical Layer Collision Avoidance
RMII	Reduced Media Independent Interface

2.0 TEST PLATFORM RASPBERRY PI 4

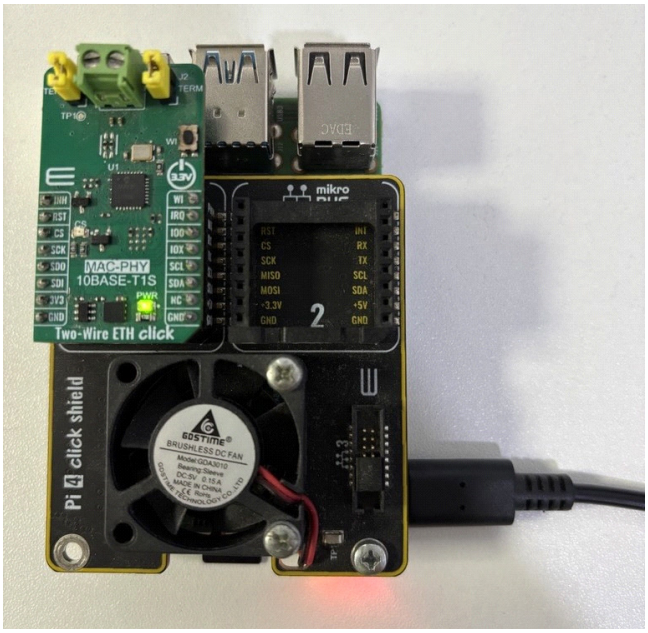
2.1 Test Setup Preparation

1. Install the Raspberry Pi Operating System on the Raspberry Pi device.
   We recommend to use the Raspberry Pi Imager for this process. For details, refer to the following link: https://www.raspberrypi.com/documentation/computers/getting-started.html#installing-the-operating-system.

2. Connect the Raspberry Pi 4 click shield board onto the 40-pin header of the Raspberry Pi board.

3. Connect the Two-Wire ETH Click board with the LAN8651 on mikroBus™ 1 of the Raspberry Pi 4 click shield board.

4. Power on the Raspberry Pi.



3.0 DRIVER INTEGRATION

The driver can be integrated into the system using one of the following methods:

- Built-in kernel support: The kernel is compiled with the LAN865x driver enabled. In this configuration, the driver becomes an integral part of the kernel image and is available immediately during system initialization.
  If you use this method, follow the procedure described in Chapter 4.0, then proceed with Chapter 5.0. After this follow the steps as outlined in Chapter 7.0 and continue with the subsequent steps.
- Loadable Kernel Module (LKM): The LAN865x driver is compiled as a separate module that can be dynamically loaded into or removed from the running kernel.
  This method provides greater flexibility, as it does not require kernel recompilation.
  If you use this method, follow the procedure outlined in Chapter 6.0, then proceed with Chapter 7.0 and continue with the subsequent steps.

4.0 PREPARE LINUX KERNEL WITH LAN865X DRIVER SUPPORT

4.1 Linux Kernel Compilation

1. Install the essential packages required to compile the kernel.

   ```
   sudo apt-get install build-essential flex bison libncurses-dev libssl-dev libelf-dev
   ```

2. Make sure the “current date and time” of Raspberry Pi are up-to-date.

   ```
   $ date
   ```

   If date and time are not up-to-date, set them manually; see the following example:

   ```
   $ sudo date -s "Friday 04 April 2025 10:44:43 AM IST"
   ```

3. Download the latest and stable Raspberry Pi kernel source code.

   ```
   $ git clone --depth=1 https://github.com/raspberrypi/linux
   ```

4. Download the latest LAN865x MAC-PHY driver files to your local system.
   a) `$ wget https://raw.githubusercontent.com/raspberrypi/linux/refs/heads/rpi-6.13.y/drivers/net/ethernet/microchip/lan865x/lan865x.c`
   b) `$ wget https://raw.githubusercontent.com/raspberrypi/linux/refs/heads/rpi-6.13.y/drivers/net/phy/microchip_t1s.c`
   c) `$ wget https://raw.githubusercontent.com/raspberrypi/linux/refs/heads/rpi-6.13.y/drivers/net/ethernet/oa_tc6.c`
   d) `$ wget https://raw.githubusercontent.com/raspberrypi/linux/refs/heads/rpi-6.13.y/include/linux/oa_tc6.h`

5. Replace the existing LAN865x MAC-PHY driver files in the Raspberry Pi kernel source code directory by copying the latest version of the LAN865x MAC-PHY driver files that you have downloaded.
   a) `$ cp lan865x.c linux/drivers/net/ethernet/microchip/lan865x/`
   b) `$ cp microchip_t1s.c linux/drivers/net/phy/`
   c) `$ cp oa_tc6.c linux/drivers/net/ethernet/`
   d) `$ cp oa_tc6.h linux/include/linux/`

   Note: The above git clone command downloads the Raspberry Pi kernel source code corresponding to Linux kernel version 6.12. However, support for the latest MAC-PHY—that is LAN865x Rev B1— is available only from Linux kernel version 6.13 onwards. Therefore, you need to download the latest LAN865x driver files from the rpi-6.13.y branch of the Raspberry Pi kernel source code and copy them into the version 6.12 directory.
   If you are using LAN865x Rev B0, you may skip steps 4 and 5 above, and proceed with the subsequent instructions.

6. Create a configuration file (.config) with default configuration.
   a) `$ cd linux`
   b) `$ KERNEL=kernel8`
   c) `$ make bcm2711_defconfig`

4.2 Enable LAN865x Driver Support

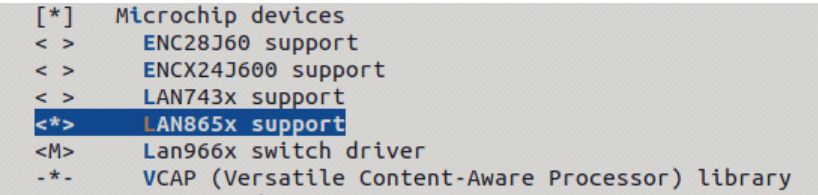
1. Configure LAN865x driver support in the kernel.

   ```
   $ make menuconfig
   ```

   The kernel configuration window opens.

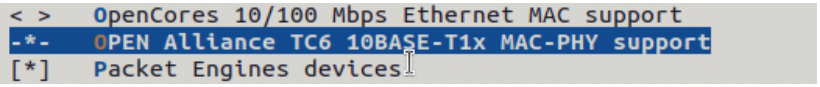
2. Enable MAC driver support.

   Select "*Device Drivers >> Network device support >> Ethernet driver support >> Microchip devices >> LAN865x support*".

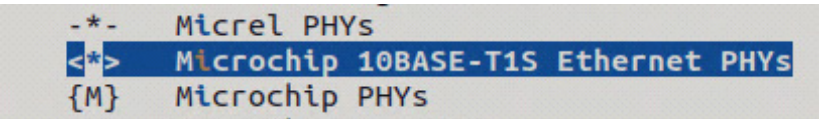
   Note: Make sure there is a "<*>" next to "LAN865x support".
   If not, select the entry and press the space bar to cycle through the options.
   The "<*>" means the drivers will be built into the kernel.

   When you click "LAN865x support", Open Alliance TC6 10BASE-T1S support will be automatically enabled.

3. Enable PHY driver support.

   Select "*Device Drivers >> Network device support >> PHY Device support and infrastructure >> Microchip 10BASE-T1S Ethernet PHYs*".

   Note: Make sure there is a "<*>" next to "Microchip 10BASE-T1S Ethernet PHYs".
   If not, select the entry and press the space bar to cycle through the options.
   The "<*>" means the drivers will be built into the kernel.

4. Save the configuration.

5. Compile the kernel.

   ```
   $ make -j6 Image.gz modules dtbs
   ```

4.3 Install Kernel

1. Install the kernel modules onto the boot media of the Raspberry Pi.

   ```
   $ sudo make -j6 modules_install
   ```

2. Create a backup image of the current kernel, install the fresh kernel image, overlays, README and unmount the partitions.

   a) `$ sudo cp /boot/firmware/$KERNEL.img /boot/firmware/$KERNEL-backup.img`

   b) `$ sudo cp arch/arm64/boot/Image.gz /boot/firmware/$KERNEL.img`

   c) `$ sudo cp arch/arm64/boot/dts/broadcom/*.dtb /boot/firmware/`

   d) `$ sudo cp arch/arm64/boot/dts/overlays/*.dtb* /boot/firmware/overlays/`

   e) `$ sudo cp arch/arm64/boot/dts/overlays/README /boot/firmware/overlays/`

3. Reboot the Raspberry Pi and run the freshly compiled kernel.

   ```
   $ sudo reboot
   ```

5.0 CONFIGURE DEVICE TREE

5.1 Add LAN865x Device Support

In your device tree's SPI node add the LAN865x device details as follows:

```
spi {

        ethernet@0 {

        compatible = "microchip,lan8651", "microchip,lan8650";

        reg = <0>;

        pinctrl-names = "default";

        pinctrl-0 = <&eth0_pins>;

        interrupt-parent = <&gpio>;

        interrupts = <6 IRQ_TYPE_EDGE_FALLING>;

        local-mac-address = [04 05 06 01 02 03];

        spi-max-frequency = <15000000>;

        };

};
```

More information about the device tree can be found at https://github.com/torvalds/linux/blob/master/Documentation/devicetree/bindings/net/microchip%2Clan8650.yaml.

5.2 Example Device Tree Configuration for a Single LAN865x Device

1. To include the LAN865x device tree overlay, open the *config.txt* file with super user access.
   ```
   $ sudo vim /boot/firmware/config.txt
   ```
   Note: If vim editor is not installed in your system, install it.
   ```
   $ sudo apt-get install vim
   ```
2. Uncomment the line `#dtparam=spi=on --> dtparam=spi=on`
3. Add the line `dtoverlay=lan865x` after the above line, so the *config.txt* should have the below lines also.
   - `dtparam=spi=on`
   - `dtoverlay=lan865x`
4. Make sure the device tree compiler (dtc) is installed on the Raspberry Pi. If not, use the following command to install it.
   ```
   $ sudo apt-get install device-tree-compiler
   ```
5. Create a device tree (dts) file.
   ```
   $ touch lan865x-overlay.dts
   ```
6. Open the file in vim editor.
   ```
   $ vim lan865x-overlay.dts
   ```
7. Add the below contents to the file.

```
// Overlay for the Microchip LAN865X Ethernet Controller
/dts-v1/;
/plugin/;

/ {
      compatible = "brcm,bcm2835";

      fragment@0 {
        target = <&spi0>;
        __overlay__ {
          /* needed to avoid dtc warning */
          #address-cells = <1>;
          #size-cells = <0>;

          status = "okay";

          /* Settings for the lan865x click board connected with Mikro Bus 1 */
          eth1: lan865x@0{
            compatible = "microchip,lan8651", "microchip,lan8650";
            reg = <0>; /* CE0 */
            pinctrl-names = "default";
            pinctrl-0 = <&eth1_pins>;
            interrupt-parent = <&gpio>;
            interrupts = <6 0x2>; /* 0x2 - falling edge trigger */
            local-mac-address = [04 05 06 01 02 03];
```

```
            spi-max-frequency = <15000000>;
            status = "okay";
          };
        };
      };

      fragment@1 {
        target = <&spidev0>;
        __overlay__ {
          status = "disabled";
        };
      };

      fragment@2 {
        target = <&gpio>;
        __overlay__ {
          eth1_pins: eth1_pins {
            brcm,pins = <6>;
            brcm,function = <0>; /* in */
            brcm,pull = <0>; /* none */
            };
          };
        };

        __overrides__ {
          int_pin_1 = <&eth1>, "interrupts:0",
                      <&eth1_pins>, "brcm,pins:0";
          speed_1   = <&eth1>, "spi-max-frequency:0";
          };
  };
```

8. Save and close the editor.
9. Compile the device tree overlay file *lan865x-overlay.dts* to generate the *lan865x.dtbo*.

   ```
   $ dtc -I dts -O dtb -o lan865x.dtbo lan865x-overlay.dts
   ```

10. Copy the generated *lan865x.dtbo* file into the */boot/overlays/* directory

    ```
    $ sudo cp lan865x.dtbo /boot/overlays/
    ```

11. Reboot the Raspberry PI.

    ```
    $ sudo reboot
    ```

5.3 Example Device Tree Configuration for Two LAN865x Devices

For this example it is necessary to connect a second Two-Wire ETH Click board with the LAN865x on mikroBus 2 of the Raspberry Pi 4 click shield board. The configuration for the two LAN865x approach is the same as described in Section 5.2, apart from the description of the *lan865x-overlay.dts* file, which now describes the second LAN865x device in addition. Therefore, you can follow the instructions given in Section 5.2, steps 1-6 and 8-11 by exchanging step 7 with the information shown below.

```
// Overlay for the Microchip LAN865X Ethernet Controller
/dts-v1/;
/plugin/;

/ {
      compatible = "brcm,bcm2835";

      fragment@0 {
        target = <&spi0>;
        __overlay__ {
          /* needed to avoid dtc warning */
          #address-cells = <1>;
          #size-cells = <0>;

          status = "okay";

          /* Settings for the lan865x click board connected with Mikro Bus 2 */
          eth2: lan865x@1{
            compatible = "microchip,lan8651", "microchip,lan8650";
            reg = <1>; /* CE1 */
            pinctrl-names = "default";
            pinctrl-0 = <&eth2_pins>;
            interrupt-parent = <&gpio>;
            interrupts = <26 0x2>; /* 0x2 - falling edge trigger */
            local-mac-address = [14 15 16 11 12 13];
            spi-max-frequency = <15000000>;
            status = "okay";
          };
          /* Settings for the lan865x click board connected with Mikro Bus 1 */
          eth1: lan865x@0{
            compatible = "microchip,lan8651", "microchip,lan8650";
            reg = <0>; /* CE0 */
            pinctrl-names = "default";
            pinctrl-0 = <&eth1_pins>;
            interrupt-parent = <&gpio>;
            interrupts = <6 0x2>; /* 0x2 - falling edge trigger */
            local-mac-address = [04 05 06 01 02 03];
            spi-max-frequency = <15000000>;
            status = "okay";
          };
        };
      };
```

```
        fragment@1 {
          target = <&spidev0>;
          __overlay__ {
            status = "disabled";
              };
          };

        fragment@2 {
          target = <&gpio>;
          __overlay__ {
            eth1_pins: eth1_pins {
              brcm,pins = <6>;
              brcm,function = <0>; /* in */
              brcm,pull = <0>; /* none */
              };
            eth2_pins: eth2_pins {
              brcm,pins = <26>;
              brcm,function = <0>; /* in */
              brcm,pull = <0>; /* none */
              };
            };
          };

          __overrides__ {
            int_pin_1 = <&eth1>, "interrupts:0",
                        <&eth1_pins>, "brcm,pins:0";
            speed_1   = <&eth1>, "spi-max-frequency:0";
            int_pin_2 = <&eth2>, "interrupts:0",
                        <&eth2_pins>, "brcm,pins:0";
            speed_2   = <&eth2>, "spi-max-frequency:0";

            };
    };
```

6.0 LOAD LAN865X DRIVER MANUALLY INTO KERNEL

Note: The steps described in this chapter have been tested on the Raspberry Pi 4 platform with kernel version 6.6.51.

6.1 Prerequisites

Note: If the Raspberry Pi OS has been freshly installed, the following prerequisites are mandatory before using the driver package. Please ensure that the Raspberry Pi is connected to the Internet, as some dependencies will need to be installed.

1. Make sure the “current date and time” of Raspberry Pi are up-to-date.

   ```
   $ date
   ```

   If date and time are not up-to-date, set them manually; see the following example:

   ```
   $ sudo date -s "Friday 04 April 2025 10:44:43 AM IST"
   ```

2. Update “apt-get”.

   ```
   $ sudo apt-get update
   ```

3. Install Linux headers.

   ```
   $ sudo apt-get install raspberrypi-kernel-headers
   ```

6.2 Configure Raspberry Pi 4

To configure the RPi 4 follow the steps described in Section 5.2.

6.3 Load the Driver

1. Create a new directory.
   a) `$ mkdir driver_setup`
   b) `$ cd driver_setup`

2. Download the latest LAN865x MAC-PHY driver files from mainline kernel source code.
   a) `$ wget https://raw.githubusercontent.com/torvalds/linux/refs/heads/master/drivers/net/ethernet/microchip/lan865x/lan865x.c`
   b) `$ wget https://raw.githubusercontent.com/torvalds/linux/refs/heads/master/drivers/net/phy/microchip_t1s.c`
   c) `$ wget https://raw.githubusercontent.com/torvalds/linux/refs/heads/master/drivers/net/ethernet/oa_tc6.c`
   d) `$ wget https://raw.githubusercontent.com/torvalds/linux/refs/heads/master/include/linux/oa_tc6.h`

3. Open the *lan865x.c* file.

   ```
   $ vim lan865x.c
   ```

4. Change the below line from

   ```
   #include <linux/oa_tc6.h>
   ```

   into

   ```
   #include “oa_tc6.h”.
   ```

5. Open the *oa_tc6.c* file.
   ```
   $ vim oa_tc6.c
   ```
6. Change the below line from
   ```
   #include <linux/oa_tc6.h>
   ```
   into
   ```
   #include "oa_tc6.h".
   ```
7. Add the following line.
   ```
   #define MDIO_MMD_POWER_UNIT 13 /* PHY POWER UNIT */
   ```
8. Create a Makefile.
   ```
   $ touch Makefile
   ```
9. Open the file in vim editor.
   ```
   $ vim Makefile
   ```
10. Add the below contents into the editor.

```
ifndef KDIR
    KDIR=/lib/modules/$(shell uname -r)/build
endif

obj-m := microchip_t1s.o
obj-m += lan865x_t1s.o
lan865x_t1s-y := lan865x.o oa_tc6.o

all:
     $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
     $(MAKE) -C $(KDIR) M=$(PWD) clean
```

11. Save and close the editor.
12. Compile the driver.
    ```
    $ make
    ```
13. Create a *load.sh* script file.
    ```
    $ touch load.sh
    ```
14. Open the file in vim editor.
    ```
    $ vim load.sh
    ```
15. Add the below contents.

```
#!/usr/bin/env bash

sudo insmod microchip_t1s.ko
sudo insmod lan865x_t1s.ko

echo performance | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor > /dev/
null
```

16. Save and close the editor.
17. Run the below command.

```
$ chmod +x load.sh
```

18. Load the driver using the *load.sh* script.

```
$ ./load.sh
```

6.4 Init Script

You can avoid manual driver loading at every boot. For this, add the program to be run on boot to the */etc/init.d* directory. This directory contains scripts that are executed during the boot process.

1. Navigate to the *init* directory and create a new script file.
   a) `$ cd /etc/init.d/`
   b) `$ sudo touch lan865x_init`
2. Open the file in vim editor.
   `$ sudo vim lan865x_init`
3. Add the below contents to the file.

```
#! /bin/sh
### BEGIN INIT INFO
# Provides: lan865x_init
# Required-Start: $remote_fs $syslog
# Required-Stop: $remote_fs $syslog
# Default-Start: 2 3 4 5
# Default-Stop: 0 1 6
# Short-Description: Load LAN865x driver
# Description: Load LAN865x MAC-PHY driver
### END INIT INFO

DRIVER_PATH="/home/pi/driver_setup/"

case "$1" in
  start)
    cd $DRIVER_PATH
    ./load.sh
    cd -
    ;;
  stop)
    ;;
  restart)
    $0 stop
    $0 start
    ;;
  *)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
    ;;
esac

exit 0
```

4. Save and close the editor.
5. Make the script as executable.
   `$ sudo chmod +x lan865x_init`

6. Run the below command.

```
$ sudo update-rc.d lan865x_init defaults
```

Note: Make sure that DRIVER_PATH variable values have been set properly.

DRIVER_PATH: Full path to the *load.sh* script file that is added in Section 6.3.

7.0 DEVICE IDENTIFICATION

The device name is required to configure the network interface using the “ethtool” command line utility.

1. Plug the LAN865x device into the system to see its device name.
2. Load driver modules into the kernel in case you are loading them manually.
3. Load the device tree overlay file (*lan865x-overlay.dts*).
4. Run the below command.

   ```
   $ ip link show
   ```

   Running the command displays detailed information about all available network interfaces. The device name varies depending on the platforms which you are using.

   For Raspberry Pi devices the *ethX* format is used. Where X is the number starting from 0. For example, the onboard Ethernet is typically “eth0”, and LAN865x may appear as “eth1”.

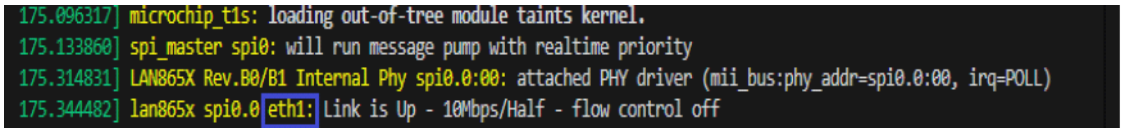
7.1 Correct Device Name Identification

When multiple devices are connected to the system, identifying a specific device using the `ip link show` command can be challenging. Instead proceed as follows:

Run the following command after inserting the LAN865x device in the system.

```
$ dmesg
```

This displays messages about the device recently connected to the system.



8.0 NETWORK INTERFACE CONFIGURATION

To manage and configure the network properties of the LAN865x, the “ethtool” command line utility is used.

8.1 “ethtool” Installation

Note:	The “ethtool” version should be 6.7 or later, since older versions do not support PLCA settings.

8.1.1 Prerequisites

Note:	Please ensure that the following dependencies are installed prior to installing “ethtool” on Linux kernel version 6.6.x and 6.12.x running on a Raspberry Pi 4 Model B. Note also that these dependencies may vary depending on the distribution used.

1. Run automake command.

```
$ sudo apt-get install automake
```

2. Install *libmnl* libraries.
   a) `$ wget https://www.netfilter.org/projects/libmnl/files/libmnl-1.0.3.tar.bz2`
   b) `$ tar -xjvf libmnl-1.0.3.tar.bz2`
   c) `$ cd libmnl-1.0.3`
   d) `$ ./configure`
   e) `$ make`
   f) `$ sudo make install`

8.1.2 Install “ethtool”

1. Clone the repository.
   a) `$ git clone https://git.kernel.org/pub/scm/network/ethtool/ethtool.git`
   b) `$ cd ethtool`
2. Run commands.
   a) `$ autoupdate`
   b) `$ ./autogen.sh`
   c) `$ ./configure --prefix=/usr --sbindir=/usr/sbin`
   d) `$ make`
   e) `$ sudo make install`
3. Make a reboot.
4. Check the “ethtool” version.

```
$ ethtool  --version
```

8.2 Configure Ethernet Devices Using “ethtool”

Run the following commands.

a) `$ sudo nmcli con add con-name t1s-static ifname eth1 type ethernet ip4 192.168.10.11/24`

b) `$ sudo nmcli con up t1s-static`

c) `$ sudo ethtool --set-plca-cfg eth1 enable on node-id 0 node-cnt 8 to-tmr 0x20 burst-cnt 0x0 burst-tmr 0x80`

d) `$ sudo ethtool --get-plca-cfg eth1`

Note related to step a: `con-name` can be anything. But remember the name for future use.
To identify the `ifname`, refer the Chapter 7.0.

Note related to step b: `t1s-static` is set as `con-name` used in the previous command.

Note: Instead of setting PLCA configurations manually every time, you can use the PLCA Configurator tool, which is described in Chapter 9.0.

9.0 PLCA CONFIGURATOR TOOL

Linux-based systems currently do not retain PLCA settings for network devices. Each time LAN865x network devices are unplugged and reconnected, or the system is rebooted, you need to reconfigure the PLCA settings by using the “ethtool” application. The PLCA Configurator tool can be utilized to automatically configure PLCA settings when the device is detected on a Linux system.

9.1 Prerequisites

- Linux kernel version should be 6.6 or later
- “ethtool” version should be 6.7 or later

9.2 Test Platform and Devices

Test platform:	Raspberry Pi 4 Model B
Kernel version:	6.6.51 6.12.25
“ethtool” version:	6.14

9.3 Install and Run the PLCA Configurator

1. Download the PLCA Configurator tool package.

   ```
   $ git clone https://github.com/MicrochipTech/linux-auto-ethtool-plca-config
   ```

2. Go to the *linux-auto-ethtool-plca-config* directory.

   ```
   $ cd linux-auto-ethtool-plca-config/
   ```

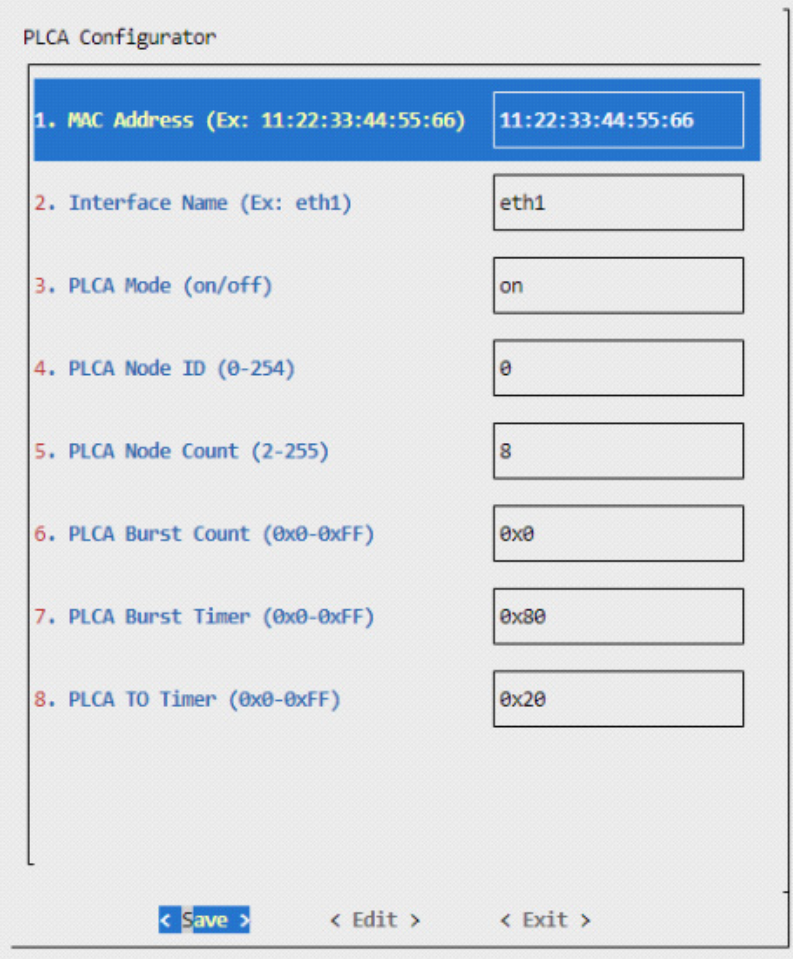
3. Run the below commands.

   ```
   $ chmod +x plca_configurator_tui.sh
   $ ./plca_configurator_tui.sh
   ```

   As the tool is based on the dialog package, it will ask for your permission to install the package if it is not already installed. You may select “Yes” if you want the tool to install the package automatically before starting the application.

   A terminal user interface appears.

4. Enter your desired settings.



5. Click **Save**.
6. Unplug and reconnect the LAN865x device (if applicable) or reboot the system, to apply the new settings.

10.0 NETWORK PERFORMANCE TEST

To test the throughput between the nodes, “Iperf3” is used.

Prepare a two-node setup and perform the test by running the following commands on the respective nodes.

Node 1: Server

Example: `$ iperf3 -s  -i 1 -p 5001`

Node 2: Client

`$ iperf3 -c <server-ip-addr> -u -b 10M -i 1 -p 5001`

Example: `iperf3 -c 192.168.10.12 -u -b 10M -i 1 -p 5001`

Based on the test results, the maximum bandwidth achieved is 9.43 Mbps.

APPENDIX A: REVISION HISTORY

Revision	Date	Section/Figure/Entry	Correction
DS00005990C	10-08-2025	Chapter 3.0	Added new chapter with information on driver integration
		Section 6.3, Step 15	Removed the following lines from the code: • `sudo ip addr add dev eth1 192.168.5.100/24` • `sudo ip link set eth1 up`
		Section 6.4, Step 3	Removed the following line from the code: `INTERFACE_NAME="eth1"`
		Section 8.2, Step a	• Changed code from "`$ sudo ip addr add dev eth1 192.168.10.11/24`" to "`$ sudo nmcli con add con-name t1s-static ifname eth1 type ethernet ip4 192.168.10.11/24`" • Added note
		Section 8.2, Step b	• Changed code from "`$ sudo ip link set eth1 up`" to "`$ sudo nmcli con up t1s-static`" • Added note
DS00005990B	07-30-2025	Section 5.2, Step 9	Changed "`$ dtc -I dts -O dtb -o lan865x.dtbo dts/lan865x-overlay.dts`" to "`$ dtc -I dts -O dtb -o lan865x.dtbo lan865x-overlay.dts`"
		Section 6.3, Step 5	Changed "oa_tc6.h" to "oa_tc6.c"
DS00005990A	05-28-2025	Initial version of this document	

Linux® is the registered trademark of Linus Torvalds in the U.S. and other countries.

Raspberry Pi® is a registered trademark of The Raspberry Pi Foundation.

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at https://www.microchip.com/en-us/about/legal-information/microchip-trademarks.

ISBN: 979-8-3371-2006-5

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at https://www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP "AS IS". MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip product is strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.